

Part 4: Planning → Building Workflow

Teaching Agentic Coding Tools to Students

Graduate Coursework Research

University of South Carolina

January 23, 2026

Outline

- 1 Why Planning Matters
- 2 The Research-Plan-Implement-Review Pattern
- 3 Plan Mode in Different Tools
- 4 Plan-Act-Reflect Framework
- 5 Static vs Dynamic Decomposition
- 6 Summary

Why Planning Matters Before Building

Typical Developer Workflow



Better Workflow



Planning saves TIME and MONEY

Key Questions

- What are real costs of skipping planning?
- How does planning improve code quality?
- How do agentic tools amplify benefits?

Discussion

Think about the last time you started coding without a plan. What went wrong?

The Cost of Skipping Planning

Wasted Effort

- Code written but never used (~30%)
- Rework and refactoring cycles
- Dead ends and false starts

Quality Issues

- More bugs (80% logic errors)
- Harder to test
- Difficult to maintain

Architectural Problems

- Structural incompatibilities
- Poor scalability decisions
- Tight coupling and dependencies

Team Communication

- Unclear requirements
- Wrong assumptions
- Conflicting implementations

With Agentic Tools

Without planning, agents amplify waste—generating pages of incorrect code in seconds.

How Planning Improves Results

Time	No Planning: 100% coding → 40% wasted With Planning: 20% plan + 20% code + 5% waste Result: 35% faster delivery
Quality	No Planning: 50-80 bugs per 1000 lines With Planning: 5-15 bugs per 1000 lines Result: 5-10x fewer production bugs
Architecture	No Planning: Major rewrites common (20%) With Planning: Minimal structural changes (5%) Result: Stable foundations
Team	No Planning: Conflicting implementations With Planning: Everyone builds same vision Result: Faster reviews, fewer conflicts

Research Evidence for Planning Benefits

NASA Software Quality Data

- With design reviews: 0.5 defects/KLOC
- Without design phase: 4.5 defects/KLOC
- **Result: 9x more defects without planning**

Microsoft Code Review (2016)

- With design discussions: 12% revision rate
- Without discussions: 43% revision rate
- **Result: 3.6x more revisions**

Google Large-Scale Changes

- With 1-2 week planning: 95% first-submit acceptance
- Without planning: 62% acceptance
- **Result: 3x higher acceptance rate**

Stack Overflow Survey 2024

- Developers who plan: 23% higher satisfaction
- Features delivered 30% faster
- Less frustration and rework

Planning Anti-patterns

1. Over-Planning

60% planning, 40% implementation
Creates outdated specs, kills flexibility

2. Planning in Isolation

One person plans, rest code
Misses practitioner input and feasibility

3. Planning Without Research

Assume domain knowledge
Miss existing solutions and constraints

4. Static Plans

Plan once, follow forever
Cannot adapt to discoveries

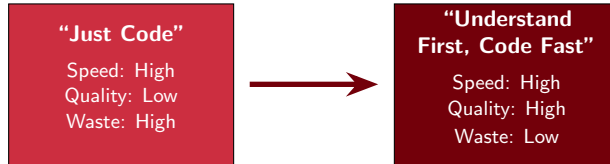
5. No Validation

Plan but never test feasibility
Discover problems during implementation

With Agentic Tools

These anti-patterns are amplified because agents execute plans rapidly—bad plans get implemented at scale.

The Planning-Building Mindset

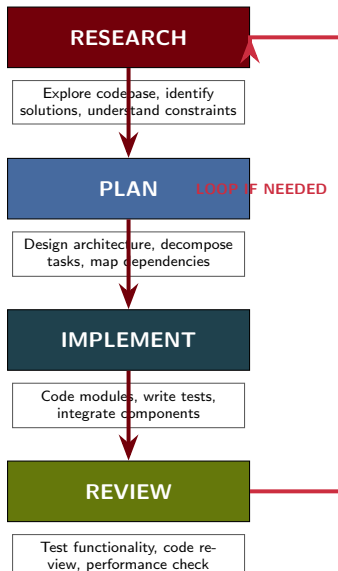


With Agentic Tools:
30 min planning → 2 hours coding
vs. 0 min planning → 5 hours coding + rework
Agent-generated code amplifies both benefits and costs

Time Allocation Target

10-20% planning • 70-80% implementing • 5-10% reviewing

The Research-Plan-Implement-Review Cycle



With Agentic Tools:

Research: Agent explores codebase, developer guides

Plan: Developer writes plan, agent reviews feasibility

Implement: Agent codes from plan, developer validates

Review: Agent and developer validate together

Research Phase: Understanding Your Problem

1. Understand the Domain

- What problem are you solving?
- Who is the user?
- What are the constraints?
- What's the success criteria?

2. Explore the Codebase

- How is related code structured?
- What patterns and conventions?
- What can you reuse?

3. Identify Constraints

- Performance requirements
- Scalability needs
- Security considerations

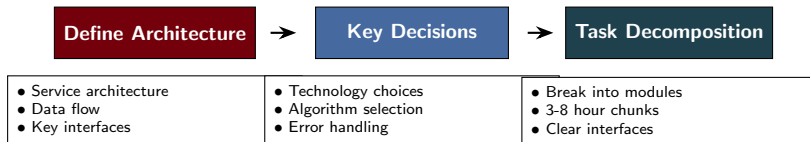
Agent's Role in Research

- Search codebase for patterns
- Find similar implementations
- Identify relevant code sections
- Verify your assumptions

Red Flags During Research

Conflicting requirements not identified • Missing critical constraints • Overlooked existing solutions

Planning Phase: Designing Your Solution



OUTPUT: Architecture Document
1. System diagram showing components 2. Data flow showing how data moves 3. Module descriptions and interfaces 4. Dependencies between modules 5. Key decisions and rationales

Detailed Task Decomposition

Example: API Request Caching Layer (~40h)

Phase 1: Setup (12h)

- Task 1.1: Design architecture (2h)
- Task 1.2: Testing infrastructure (3h)
- Task 1.3: Dependency setup (2h)
- Task 1.4: Documentation (5h)

Phase 2: Core Implementation (20h)

- Task 2.1: Cache interface (3h)
- Task 2.2: Redis implementation (5h)
- Task 2.3: Invalidation system (4h)
- Task 2.4: Performance optimization (5h)
- Task 2.5: Monitoring (3h)

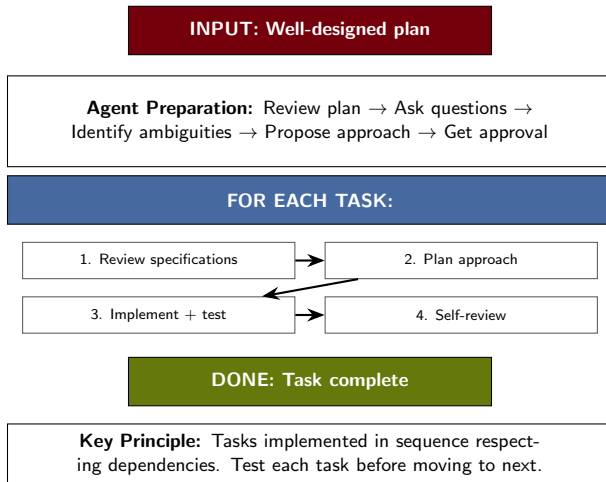
Key Principles

- 1 Independence:** Minimal dependencies
- 2 Size:** 3-8 hour chunks
- 3 Interfaces:** Clear inputs/outputs
- 4 Testability:** Can test independently

Red Flags

- Tasks >12 hours (too big)
- Tasks <1 hour (too granular)
- Circular dependencies
- Unclear “done” criteria

Implementation Phase: Building from Plans



Review Phase: Validation and QA

Testing Levels

Level 1: Unit

Each module in isolation

Level 2: Integration

Modules tested together

Level 3: System

Full end-to-end testing

Level 4: Acceptance

Meets all requirements?

Review Checklist

- ☐ All requirements done
- ☐ Edge cases handled
- ☐ Test coverage >80%
- ☐ Code follows standards
- ☐ API documented
- ☐ Performance acceptable
- ☐ No security issues

Common Issues

- Missing edge cases
- Poor error messages
- Untested code paths
- Performance issues
- Security gaps

Review Questions to Ask

“What happens if this fails?” • “Can this be called with bad input?” • “What’s worst-case performance?” • “Will the next developer understand this?”

Complete Workflow Example: JWT Authentication

Research Phase (2h)

- Found session-based auth exists
- Identified backward compatibility need
- Found existing error patterns
- Clarified JWT requirements

Planning Phase (2h)

- Designed JWT service architecture
- Key decision: Use PyJWT library
- Task breakdown: 5 tasks, 16h total

Implementation (16h)

- Task 1: JWT Service (4h) ✓
- Task 2: Validation Middleware (3h) ✓
- Task 3: Refresh Endpoint (2h) ✓
- Task 4: Update Endpoints (3h) ✓
- Task 5: Tests + Docs (4h) ✓

Review Phase (3h)

- 47 unit tests, 92% coverage
- 23 integration tests passing
- All requirements verified

Common Pitfalls and How to Avoid Them

Pitfall: Skipping Research

Symptom: “I know this codebase”

Risk: Miss existing solutions

Fix: Always spend 10-20% on research

Pitfall: Planning Too Much

Symptom: 2-week plan for 1-week work

Risk: Plan becomes outdated

Fix: Plan just enough to start

Pitfall: Tasks Too Big

Symptom: “Task 1: Implement entire system”

Risk: Can't test incrementally

Fix: Break into 3-8 hour chunks

Pitfall: No Acceptance Criteria

Symptom: “Task: implement caching”

Risk: Unclear when done

Fix: Define measurable criteria

Pitfall: Missing Test Planning

Symptom: Tests thought of after

Risk: Untestable code

Fix: Plan tests with tasks

Pitfall: Agent Without Review

Symptom: Agent codes for 2h unreviewed

Risk: Wrong direction

Fix: Review frequently, feedback early

How Different Agentic Tools Support Planning

Agentic Tools Planning Capabilities		
Tool	Planning Mode	Strength
Claude Code	Explicit planning before coding	Detailed analysis
Cursor	Plan mode with Composer	IDE-integrated
Aider	Architect mode for design	Focused design
Codex CLI	Read-only planning phase	Research oriented
GitHub Copilot	Chat-based planning	Interactive

Traditional Mode:
Agent → Code immediately
Risk: Wrong direction early

Planning Mode:
Agent → Analyze → Plan → Code
Risk: Caught in plan phase

Claude Code: Explicit Planning Phase



Plan Output Format

- High-level approach
- Architecture overview
- Task breakdown with estimates
- Key decisions and rationales
- Potential risks
- Test strategy
- Questions for clarification

Best For

Large features (>20h), architectural decisions, complex integrations

Getting Better Plans from Claude Code

1. Provide Rich Context

Good: "Add authentication"

Better: "FastAPI REST API needs JWT auth. Existing code uses [pattern]. Must support web and mobile."

2. Show Existing Patterns

Good: "Follow conventions"

Better: "Here's our middleware [code], error pattern [code], testing approach [code]"

3. Specify Constraints

Good: "Must be performant"

Better: "Response <50ms, 20 DB connections max, backward compatible"

4. Define Success Criteria

Good: "Make it work"

Better: "All 87 tests pass, >80% coverage, mobile clients authenticate"

5. Request Specific Sections

Ask for: Architecture diagram, task breakdown, key decisions, risks, test strategy, dependency graph, clarifying questions

Iterate!

"This approach is too complex"

"What if we used approach B?"

"How do we mitigate this risk?"

Cursor: IDE-Integrated Planning

Cursor Composer Workflow

- 1 Open Composer mode
- 2 Select files for context
- 3 Describe requirements
- 4 Review proposed diffs
- 5 Iterate or apply changes

Strengths

- Visual context in IDE
- File-aware planning
- Interactive diffs
- Immediate testing
- Natural integration

Limitations

- Less detailed analysis
- Better for local changes
- Implicit planning
- No task breakdown
- Context window limits

Best For

- Adding functions/methods
- Refactoring existing code
- Adding validation/utilities
- Quick iterations

Aider: Terminal-Based Architecture Planning

Aider Architect Workflow

- 1 `aider --architect`
- 2 Describe the feature
- 3 Architect analyzes codebase
- 4 Proposes solution
- 5 You review and decide
- 6 Aider implements

Features

- Planning-first focus
- Terminal integration
- Version control aware
- Conversational iteration

Example Session

You: "Add rate limiting to API"

Architect: "Analysis complete. Plan:

- Create middleware
- Use Redis for counts
- Track per user/endpoint
- Config-driven limits

Should I implement?"

You: "What about anonymous users?"

Architect: "Options: A) Deny B) Generous limit C) Rate by IP"

Codex CLI: Research-Focused Planning

Read-Only Mode Features

Can Do:

- Search codebase
- Analyze code patterns
- Find similar implementations
- Understand existing approaches
- Document findings

Cannot Do:

- Write or modify code
- Commit changes
- Run tests

Research Prompts

“Find all database queries for users”
“Show me caching implementations”
“What error handling patterns exist?”
“How is validation done?”
“What are performance requirements?”

Workflow Position

Step 1: Codex research (read-only)

Step 2: Claude Code planning

Step 3: Aider/Cursor implementation

Perfect For

Research phase • Exploring unfamiliar codebases • Finding patterns • Gathering requirements early

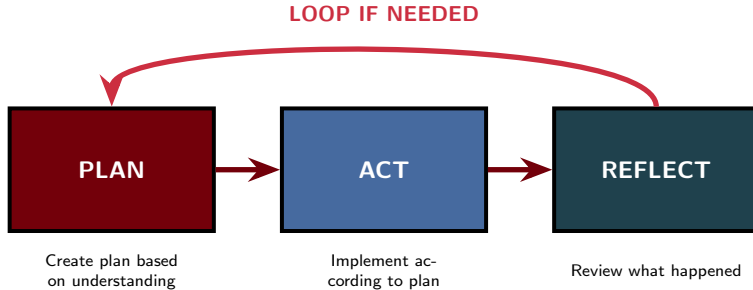
Which Tool for Which Planning Task?

Planning Task Matrix

Task	Claude	Cursor	Aider	Codex
Understand codebase	Good	Good	Good	Best
Find patterns	Good	Fair	Good	Best
Explore alternatives	Best	Fair	Best	N/A
Architecture design	Best	Fair	Good	N/A
Task decomposition	Best	Fair	Good	N/A
Visual review	Fair	Best	Fair	N/A
Quick iteration	Good	Best	Good	N/A

Recommendation: Codex for research → Claude Code for planning → Aider validation → Cursor/Claude implementation

The Plan-Act-Reflect Cycle



If plan good + implementation good: DONE, move to next feature
If issues discovered: Loop back to PLAN with new understanding

Plan-Act-Reflect: Phase Details

PLAN Phase

Input:

Requirements, constraints, patterns

Activities:

Design architecture
Break down tasks
Create test strategy
Identify risks

Output:

Architecture doc
Task breakdown
Risk analysis

ACT Phase

Input:

Approved plan, task breakdown

Activities:

Implement each task
Write tests
Verify against spec
Integrate components

Output:

Working code
Test results
Lessons learned

REFLECT Phase

Input:

Completed implementation

Activities:

Compare to plan
Analyze deviations
Review quality
Team feedback

Output:

Accept/reject decision
Improvements identified

When to Loop Back

Major issues discovered • Requirements changed • Architecture problems found • Quality unacceptable

Learning from Deviations

Deviation: Took Longer Than Planned

Why?

- Estimate too optimistic
- Unexpected complexity
- Discovered new requirements

Adjust: Future plans more conservative

Deviation: Code Quality Issues

Why?

- Plan didn't address quality
- Tests insufficient
- Code review found issues

Adjust: Increase review/test in plan

Deviation: Architecture Issues

Why?

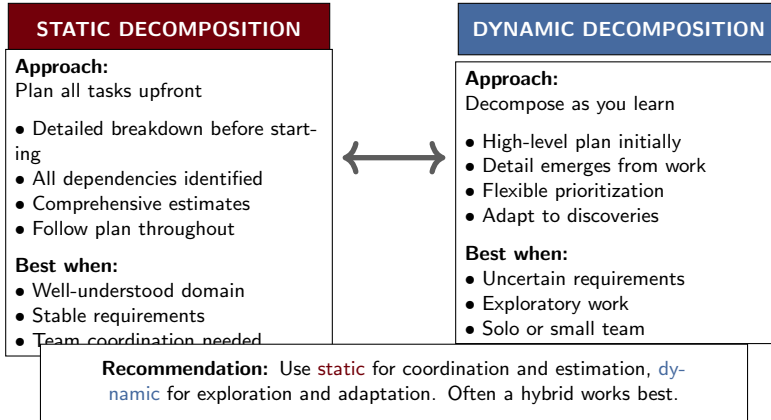
- Original understanding wrong
- Plan didn't match existing code
- Missed integration point

Adjust: More planning time and depth

Questions to Ask in Reflection

- "Are we on track?"
- "What have we learned?"
- "Do we need to adjust the plan?"
- "Are there emerging risks?"
- "Should we proceed or revise?"

Static vs Dynamic Task Decomposition



Comparing Decomposition Approaches

Dimension	Static	Dynamic
Planning time	High upfront	Spread throughout
Adaptability	Lower	Higher
Team coordination	Easier	Harder
Estimation accuracy	Better for known work	Better for unknown
Risk of over-planning	Higher	Lower
Risk of under-planning	Lower	Higher
Agentic tool fit	Good for large tasks	Good for exploration

HYBRID APPROACH (Recommended)

Static: Architecture, major milestones, external dependencies

Dynamic: Implementation details, emerging requirements, optimizations

Pattern Summary: Key Takeaways

The Pattern

Research → Plan → Implement → Review → Loop

Why It Works

- Research prevents missed context
- Planning prevents wrong implementations
- Implementation translates design to code
- Review verifies everything works
- Loop enables continuous improvement

Time Allocation

- Research: 10-15%

With Agentic Tools

- Research: Agent helps explore faster
- Planning: More detail = better agent output
- Implementation: 40% faster
- Review: More important than ever

Key Principle

Good plans → good agent code

Poor plans → poor agent code

Planning skill becomes MORE important with agentic tools, not less.

When to Use This Pattern

Always Use For

- New features (>1 day of work)
- Architecture changes
- Performance optimizations
- Integration work
- Complex bug fixes

Can Skip For

- Trivial fixes (<1 hour)
- Simple bug fixes
- Documentation updates
- Routine refactoring

Don't Skip Because of...

- “Just quick code”
- “I know this codebase”
- “Let's figure it out as we go”
- “Time pressure”

Time pressure makes planning MORE important, not less!

The Bottom Line

Planning is not the opposite of speed—it's the path to **sustainable speed**.

Questions?

Discussion Topics:

- How does planning change with agentic tools?
- What's the right balance between planning and doing?
 - How do you handle plans that need to change?
 - Which planning tool would you try first?
 - What's your biggest planning challenge?